



UNIVERSIDADE FEDERAL DO CEARÁ

Code smells em frameworks front-end

Disciplina: Qualidade de Software

Equipe: Gabriel Alves e Juan Pimentel

Projeto: Rocket.Chat

Fork: https://github.com/Juandbpimentel/Rocket.Chat_Refact

Gabriel Alves

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Any Type

No Rocket.Chat, estou removendo o uso do tipo any em diversos pontos do código, especialmente em componentes React e funções utilitárias, pois esse tipo genérico enfraquece a segurança do TypeScript e pode mascarar erros que só apareceriam em tempo de execução. O objetivo é garantir que todas as estruturas de dados, props e estados estejam devidamente tipados, aumentando a robustez e a previsibilidade do sistema.

Minhas principais dificuldades na remoção do code smell são:

A maior dificuldade tem sido identificar todos os lugares onde o any foi utilizado de forma implícita, principalmente em funções que recebem dados dinâmicos ou em componentes que lidam com múltiplos formatos de entrada. Além disso, definir tipos precisos para objetos complexos sem perder flexibilidade exige um entendimento detalhado do domínio e dos fluxos de dados do Rocket.Chat, o que demanda tempo e atenção para não introduzir quebras em funcionalidades já existentes.

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

Para resolver esse problema, estou criando tipos e interfaces nomeadas para modelar os dados de forma clara, extraindo funções de validação e transformação para isolar conversões de tipos e substituindo o any por unknown quando ainda não é possível definir o tipo final, refinando depois para algo mais preciso. Também centralizo constantes e valores padrão para evitar duplicidade e facilitar a manutenção, além de refatorar condicionais extensas para utilizar tipos união e switch statements, tornando o fluxo de decisão mais explícito e seguro.

Gabriel Alves

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Missing Union Type Abstraction

No Rocket.Chat, estou substituindo o uso direto de strings e números “mágicos” em condicionais e props por tipos união explícitos, principalmente em componentes que lidam com abas, filtros e status. Essa abordagem torna o código mais seguro e fácil de manter, pois reduz o risco de erros por digitação e facilita a identificação de todos os casos



UNIVERSIDADE FEDERAL DO CEARÁ

possíveis em cada fluxo.

Minhas principais dificuldades na remoção do code smell são:

O maior desafio tem sido mapear todas as variantes reais utilizadas no sistema, já que muitos valores estavam espalhados em diferentes arquivos e funções, além de garantir que a introdução dos tipos união não afete integrações externas ou contratos já existentes. Também é necessário ajustar testes e funções auxiliares para que aceitem apenas os valores previstos na união, evitando que casos não tratados passem despercebidos.

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

Para resolver esse problema, estou criando tipos união nomeados para cada conjunto de valores possíveis, refatorando condicionais para utilizar switch statements e centralizando regras de negócio em funções auxiliares. Dessa forma, o fluxo de decisão fica mais claro, a manutenção se torna mais simples e o código ganha em previsibilidade e segurança.

Juan Pimentel

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Overly Flexible Props

Estou tornando as props dos componentes mais restritas e explícitas, eliminando o uso de objetos genéricos e múltiplas flags booleanas que permitiam combinações inválidas e dificultavam a compreensão do componente. O objetivo é garantir que cada componente aceite apenas as combinações de props realmente válidas e representativas dos modos de uso previstos.

Minhas principais dificuldades na remoção do code smell são:

A principal dificuldade tem sido identificar todos os modos reais de uso dos componentes, já que muitos eram utilizados de formas diferentes em partes distintas do sistema. Além disso, é preciso garantir que a tipagem mais restrita não quebre funcionalidades existentes, especialmente em componentes compartilhados por diferentes módulos.

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

Para resolver esse problema, estou mapeando os modos de uso de cada componente, criando tipos união para representar essas variações de forma explícita e extraíndo subcomponentes para isolar responsabilidades visuais e lógicas. Também centralizo valores padrão em hooks auxiliares e simplifico o render, reduzindo condicionais e tornando o fluxo mais previsível e seguro.

Juan Pimentel

Eu estou atualmente trabalhando na refatoração do seguinte code smell: Enum Implicit Values



UNIVERSIDADE FEDERAL DO CEARÁ

Eu redefini os enums para que tenham valores literais explícitos e eliminando o uso de strings ou números mágicos equivalentes a enums em diferentes partes do código. Isso facilita a rastreabilidade, reduz inconsistências e torna o sistema mais robusto, principalmente em integrações com APIs externas.

Minhas principais dificuldades na remoção do code smell são:

O maior desafio tem sido localizar todos os pontos onde valores mágicos eram utilizados no lugar de enums e garantir que a mudança para enums explícitos não quebre integrações já estabelecidas ou contratos externos. Também é necessário ajustar funções auxiliares e testes para que utilizem os novos enums de forma consistente.

Eu estou usando os seguintes métodos de refatoração para remover o code smell:

Para resolver esse problema, estou redefinindo os enums com valores literais explícitos, extraindo funções auxiliares para conversão e apresentação desses valores e centralizando as definições e conversões dos enums em módulos únicos. Assim, padronizo o uso, facilito futuras manutenções e torno o código mais seguro e compreensível.